Data transformation with dplyr

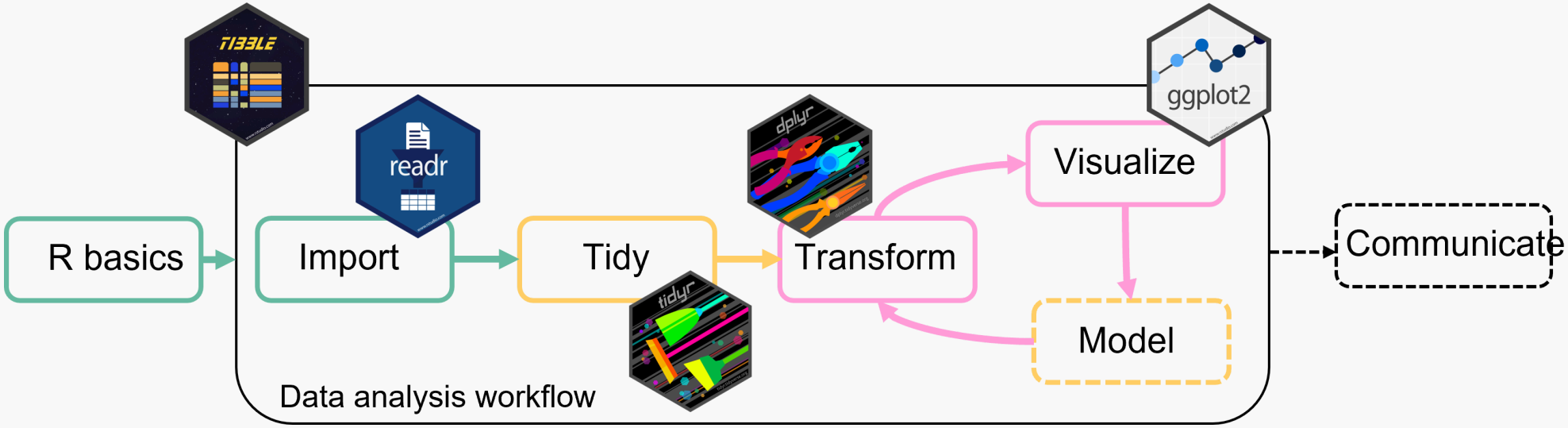
Day 2 - Introduction to Data Analysis with R

Selina Baldauf
Freie Universität Berlin - Theoretical Ecology

March 13, 2026

Data transformation

Data transformation is an important step in **understanding** the data and **preparing** it for further analysis.



We can use the tidyverse package `dplyr` for this.

The data

The `gapminder` dataset contains data on life expectancy, GDP per capita, and population for 142 countries from 1952 to 2007.

```
# install.packages("gapminder")
library(gapminder)
gapminder
#> # A tibble: 1,704 × 6
#>    country     continent  year lifeExp      pop gdpPercap
#>    <fct>       <fct>     <int>   <dbl>    <int>     <dbl>
#>  1 Afghanistan Asia       1952    28.8  8425333      779.
#>  2 Afghanistan Asia       1957    30.3  9240934      821.
#>  3 Afghanistan Asia       1962    32.0 10267083      853.
#>  4 Afghanistan Asia       1967    34.0 11537966      836.
#>  5 Afghanistan Asia       1972    36.1 13079460      740.
#>  6 Afghanistan Asia       1977    38.4 14880372      786.
#>  7 Afghanistan Asia       1982    39.9 12881816      978.
#>  8 Afghanistan Asia       1987    40.8 13867957      852.
#>  9 Afghanistan Asia       1992    41.7 16317921      649.
#> 10 Afghanistan Asia       1997    41.8 22227415      635.
#> # ℹ 1,694 more rows
```

Data transformation

With `dplyr` we can (among other things)

- **Filter** data to analyse only a part of it
- **Create** new variables
- **Summarize** data
- **Combine** multiple tables
- **Rename** variables
- **Reorder** observations or variables

To get started load the package `dplyr`:

```
library(dplyr)
# or
library(tidyverse)
```

Dplyr basic vocabulary

All of the `dplyr` functions work similarly:

- **First argument** is the data (a tibble)
- **Other arguments** specify what to do exactly
- **Return** a tibble

`filter()`

> picks rows based on their value

filter()

Filter only European countries:

```
filter(gapminder, continent == "Europe")
#> # A tibble: 360 × 6
#>    country continent  year lifeExp     pop gdpPercap
#>    <fct>   <fct>     <int>   <dbl>   <int>     <dbl>
#>  1 Albania Europe     1952    55.2 1282697     1601.
#>  2 Albania Europe     1957    59.3 1476505     1942.
#>  3 Albania Europe     1962    64.8 1728137     2313.
#>  4 Albania Europe     1967    66.2 1984060     2760.
#>  5 Albania Europe     1972    67.7 2263554     3313.
#>  6 Albania Europe     1977    68.9 2509048     3533.
#>  7 Albania Europe     1982    70.4 2780097     3631.
#>  8 Albania Europe     1987    72   3075321     3739.
#>  9 Albania Europe     1992    71.6 3326498     2497.
#> 10 Albania Europe     1997    73.0 3428038     3193.
#> # ℹ 350 more rows
```

`filter()` goes through each row and returns only rows where the `continent` is `"Europe"`

ⓘ To reuse, save result in a variable

```
gapminder_europe <- filter(gapminder, continent == "Europe")
```

`filter()`: comparison operators

- `==` equal to
- `!=` not equal to
- `<`, `>` less than, greater than
- `<=`, `>=` less than or equal to, greater than or equal to
- `%in%` in a set of values

Other examples

```
# Filter for life expectancy greater than 80
filter(gapminder, lifeExp > 80)

# All rows except European countries
filter(gapminder, continent != "Europe")

# All rows where continent is either "Africa", "Americas", or "Asia"
filter(gapminder, continent %in% c("Africa", "Americas", "Asia"))
```

`filter()`: combine conditions

Use `&` (and) - both conditions must be true:

```
filter(gapminder, continent == "Europe" & year == 2007)
#> # A tibble: 30 × 6
#>    country                continent  year lifeExp      pop gdpPercap
#>    <fct>                  <fct>     <int>   <dbl>    <int>     <dbl>
#>  1 Albania                Europe     2007    76.4  3600523     5937.
#>  2 Austria                Europe     2007    79.8  8199783    36126.
#>  3 Belgium                Europe     2007    79.4 10392226    33693.
#>  4 Bosnia and Herzegovina Europe     2007    74.9  4552198     7446.
#>  5 Bulgaria               Europe     2007    73.0  7322858    10681.
#>  6 Croatia                Europe     2007    75.7  4493312    14619.
#>  7 Czech Republic         Europe     2007    76.5 10228744    22833.
#>  8 Denmark                Europe     2007    78.3  5468120    35278.
#>  9 Finland                Europe     2007    79.3  5238460    33207.
#> 10 France                 Europe     2007    80.7 61083916    30470.
#> # ℹ 20 more rows
```

`filter()`: combine conditions

Use | (or) - one conditions must be true:

```
# Filter years outside the range 1970–2005
filter(gapminder, year >= 2005 | year <= 1970)
#> # A tibble: 710 × 6
#>    country     continent  year lifeExp      pop gdpPercap
#>    <fct>       <fct>     <int>   <dbl>    <int>     <dbl>
#>  1 Afghanistan Asia       1952    28.8  8425333      779.
#>  2 Afghanistan Asia       1957    30.3  9240934      821.
#>  3 Afghanistan Asia       1962    32.0 10267083      853.
#>  4 Afghanistan Asia       1967    34.0 11537966      836.
#>  5 Afghanistan Asia       2007    43.8 31889923      975.
#>  6 Albania     Europe     1952    55.2  1282697     1601.
#>  7 Albania     Europe     1957    59.3  1476505     1942.
#>  8 Albania     Europe     1962    64.8  1728137     2313.
#>  9 Albania     Europe     1967    66.2  1984060     2760.
#> 10 Albania     Europe     2007    76.4  3600523     5937.
#> # ℹ 700 more rows
```

`filter_out()`: opposite of filter

Relatively new function - keeps rows that **don't** match the condition:

```
# Keep rows that are NOT from Europe
filter_out(gapminder, continent == "Europe")
#> Error in `filter_out()`:
#> ! could not find function "filter_out"
```

```
# equivalent to
filter(gapminder, continent != "Europe")
```

`select()`

> picks columns based on their names

select()

Select the columns `country`, `year`, and `lifeExp`

```
select(gapminder, country, year, lifeExp)
#> # A tibble: 1,704 × 3
#>    country      year lifeExp
#>    <fct>       <int>   <dbl>
#>  1 Afghanistan  1952    28.8
#>  2 Afghanistan  1957    30.3
#>  3 Afghanistan  1962    32.0
#>  4 Afghanistan  1967    34.0
#>  5 Afghanistan  1972    36.1
#>  6 Afghanistan  1977    38.4
#>  7 Afghanistan  1982    39.9
#>  8 Afghanistan  1987    40.8
#>  9 Afghanistan  1992    41.7
#> 10 Afghanistan  1997    41.8
#> # ℹ 1,694 more rows
```

Remove variables using `-`

```
select(gapminder, -country, -year, -lifeExp)
```

`select()` + `starts_with()`

Select all columns that start with `"c"`

```
select(gapminder, starts_with("c"))
#> # A tibble: 1,704 × 2
#>    country     continent
#>    <fct>       <fct>
#>  1 Afghanistan Asia
#>  2 Afghanistan Asia
#>  3 Afghanistan Asia
#>  4 Afghanistan Asia
#>  5 Afghanistan Asia
#>  6 Afghanistan Asia
#>  7 Afghanistan Asia
#>  8 Afghanistan Asia
#>  9 Afghanistan Asia
#> 10 Afghanistan Asia
#> # ℹ 1,694 more rows
```

There are also `ends_with()`, `contains()`, and more - see `?select` for details.

`mutate()`

> Adds new columns to your data

mutate()

New columns can be added based on values from other columns

```
mutate(gapminder, gdp = gdpPercap * pop)
#> # A tibble: 1,704 × 7
#>    country     continent  year lifeExp      pop gdpPercap          gdp
#>    <fct>       <fct>     <int>   <dbl>    <int>     <dbl>        <dbl>
#>  1 Afghanistan Asia       1952    28.8  8425333      779.  6567086330.
#>  2 Afghanistan Asia       1957    30.3  9240934      821.  7585448670.
#>  3 Afghanistan Asia       1962    32.0 10267083      853.  8758855797.
#>  4 Afghanistan Asia       1967    34.0 11537966      836.  9648014150.
#>  5 Afghanistan Asia       1972    36.1 13079460      740.  9678553274.
#>  6 Afghanistan Asia       1977    38.4 14880372      786. 11697659231.
#>  7 Afghanistan Asia       1982    39.9 12881816      978. 12598563401.
#>  8 Afghanistan Asia       1987    40.8 13867957      852. 11820990309.
#>  9 Afghanistan Asia       1992    41.7 16317921      649. 10595901589.
#> 10 Afghanistan Asia       1997    41.8 22227415      635. 14121995875.
#> # ℹ 1,694 more rows
```

mutate()

Add multiple new columns at once:

```
mutate(gapminder, gdp = gdpPercap * pop, pop_million = pop / 1e6)
#> # A tibble: 1,704 × 8
#>    country     continent  year lifeExp      pop gdpPercap        gdp pop_million
#>    <fct>       <fct>     <int>   <dbl>    <int>     <dbl>      <dbl>       <dbl>
#>  1 Afghanistan Asia       1952    28.8  8425333      779.    6.57e 9        8.43
#>  2 Afghanistan Asia       1957    30.3  9240934      821.    7.59e 9        9.24
#>  3 Afghanistan Asia       1962    32.0 10267083      853.    8.76e 9       10.3
#>  4 Afghanistan Asia       1967    34.0 11537966      836.    9.65e 9       11.5
#>  5 Afghanistan Asia       1972    36.1 13079460      740.    9.68e 9       13.1
#>  6 Afghanistan Asia       1977    38.4 14880372      786.    1.17e10       14.9
#>  7 Afghanistan Asia       1982    39.9 12881816      978.    1.26e10       12.9
#>  8 Afghanistan Asia       1987    40.8 13867957      852.    1.18e10       13.9
#>  9 Afghanistan Asia       1992    41.7 16317921      649.    1.06e10       16.3
#> 10 Afghanistan Asia       1997    41.8 22227415      635.    1.41e10       22.2
#> # ℹ 1,694 more rows
```

mutate() + case_when()

Use `case_when` to add column values conditional on other columns.

```
mutate(
  gapminder,
  life_category = case_when(
    lifeExp < 50 ~ "low", # case 1
    lifeExp < 70 ~ "medium", # case 2
    lifeExp >= 70 ~ "high", # case 3
    .default = NA # all other
  )
)
#> # A tibble: 1,704 × 7
#>    country     continent  year lifeExp      pop gdpPercap life_category
#>    <fct>       <fct>     <int>   <dbl>    <int>     <dbl> <chr>
#>  1 Afghanistan Asia       1952    28.8  8425333      779. low
#>  2 Afghanistan Asia       1957    30.3  9240934      821. low
#>  3 Afghanistan Asia       1962    32.0 10267083      853. low
#>  4 Afghanistan Asia       1967    34.0 11537966      836. low
#>  5 Afghanistan Asia       1972    36.1 13079460      740. low
#>  6 Afghanistan Asia       1977    38.4 14880372      786. low
#>  7 Afghanistan Asia       1982    39.9 12881816      978. low
#>  8 Afghanistan Asia       1987    40.8 13867957      852. low
#>  9 Afghanistan Asia       1992    41.7 16317921      649. low
#> 10 Afghanistan Asia       1997    41.8 22227415      635. low
#> # ℹ 1,694 more rows
```

`drop_na()`: remove missing values

Remove rows with `NA` values in any column:

```
# Drop all rows with NAs in any column
drop_na(gapminder)
#> # A tibble: 1,704 × 6
#>    country     continent  year lifeExp      pop gdpPercap
#>    <fct>       <fct>     <int>   <dbl>    <int>     <dbl>
#>  1 Afghanistan Asia       1952    28.8  8425333      779.
#>  2 Afghanistan Asia       1957    30.3  9240934      821.
#>  3 Afghanistan Asia       1962    32.0 10267083      853.
#>  4 Afghanistan Asia       1967    34.0 11537966      836.
#>  5 Afghanistan Asia       1972    36.1 13079460      740.
#>  6 Afghanistan Asia       1977    38.4 14880372      786.
#>  7 Afghanistan Asia       1982    39.9 12881816      978.
#>  8 Afghanistan Asia       1987    40.8 13867957      852.
#>  9 Afghanistan Asia       1992    41.7 16317921      649.
#> 10 Afghanistan Asia       1997    41.8 22227415      635.
#> # ℹ 1,694 more rows
```

The gapminder data has no missing values, so `drop_na()` doesn't change anything here.

`drop_na()`: remove missing values

Remove rows with `NA` values in (a) specific column(s):

```
# Drop rows with NAs in specific columns
drop_na(gapminder, lifeExp, gdpPercap)
#> # A tibble: 1,704 × 6
#>    country     continent  year lifeExp      pop gdpPercap
#>    <fct>       <fct>     <int>   <dbl>    <int>     <dbl>
#>  1 Afghanistan Asia       1952    28.8  8425333      779.
#>  2 Afghanistan Asia       1957    30.3  9240934      821.
#>  3 Afghanistan Asia       1962    32.0 10267083      853.
#>  4 Afghanistan Asia       1967    34.0 11537966      836.
#>  5 Afghanistan Asia       1972    36.1 13079460      740.
#>  6 Afghanistan Asia       1977    38.4 14880372      786.
#>  7 Afghanistan Asia       1982    39.9 12881816      978.
#>  8 Afghanistan Asia       1987    40.8 13867957      852.
#>  9 Afghanistan Asia       1992    41.7 16317921      649.
#> 10 Afghanistan Asia       1997    41.8 22227415      635.
#> # ℹ 1,694 more rows
```

`drop_na()` is part of the `tidyr` package which is loaded with `library(tidyverse)`.

The pipe |>

Combine multiple data operations into one command

The problem: multiple steps

Data cleaning and manipulation usually requires **several steps in sequence**.

```
# Step 1: Remove missing values
gapminder_clean <- drop_na(gapminder, lifeExp)
# Step 2: Select only European countries
gapminder_clean <- filter(gapminder_clean, continent == "Europe")
# Step 3: Add a new variable
gapminder_clean <- mutate(gapminder_clean, gdp = gdpPercap * pop)
```

This works, but creates lots of intermediate steps.

The pipe |>

The pipe operator passes the result of one step as the **first argument** to the next step.

```
# This
filter(gapminder, continent == "Europe")

# is the same as
gapminder |> filter(continent == "Europe")
```

This is perfect for dplyr because:

- First argument is the data (a tibble)
- Output (return value) is also a tibble

The pipe |>

The pipe operator passes the result of one step as the **first argument** to the next step.

Instead of this:

```
# Step 1: Remove missing values
gapminder_clean <- drop_na(gapminder, lifeExp)
# Step 2: Select only European countries
gapminder_clean <- filter(gapminder_clean, continent ==
# Step 3: Add a new variable
gapminder_clean <- mutate(gapminder_clean, gdp = gdpPerc
```

We can do this:

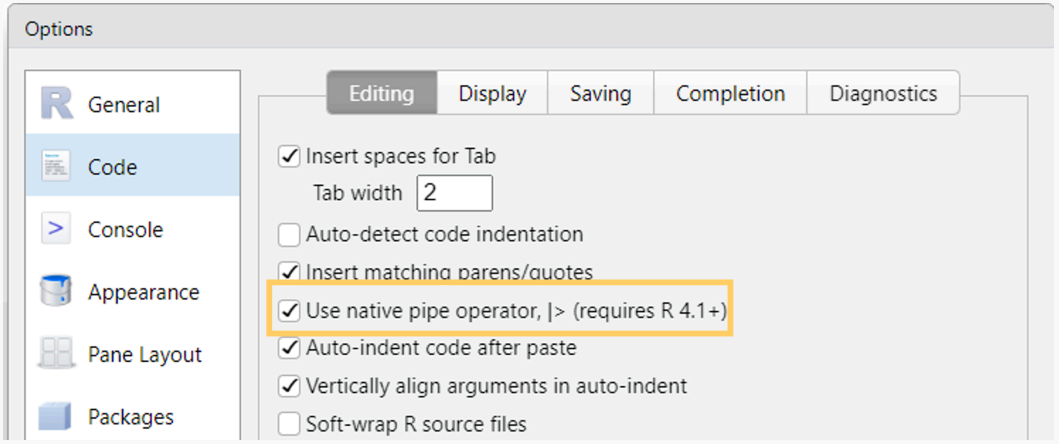
```
gapminder_clean <- gapminder |>
  drop_na(lifeExp) |>
  filter(continent == "Europe") |>
  mutate(gdp = gdpPercap * pop)
```

Read |> as **"and then"**:

Take gapminder, and then drop NAs, and then filter, and then add a column.

The pipe |>

- 2 different pipes in R (work similarly):
  - |>: base R pipe, available in R 4.1 and later
  - %>%: magrittr pipe, used in older versions and still common in tidyverse code
- I prefer |> because it's built into R
- Turn on the base R pipe in RStudio: Tools > Global Options > Code



Tip

Use the keyboard shortcut `Ctrl/Cmd + Shift + M` to insert `|>`

Now you

Task 1 (25 min)

Filter, select, and mutate the penguin data

Find the task description here

Summarizing data

From many rows to key numbers

summarize()

summarize will collapse the data to a single row

```
summarize(
  gapminder,
  mean_lifeExp = mean(lifeExp),
  mean_gdpPercap = mean(gdpPercap)
)
#> # A tibble: 1 × 2
#>   mean_lifeExp mean_gdpPercap
#>          <dbl>          <dbl>
#> 1         59.5          7215.
```

Note

The gapminder data has no missing values. With real data, use `na.rm = TRUE` to ignore NAs:

```
summarize(gapminder, mean_lifeExp = mean(lifeExp, na.rm = TRUE))
```

Otherwise `mean()` will return `NA` if any value is missing.

`summarize()` by group

`summarize` is much more useful in combination with the grouping argument `.by`

- **summary** will be calculated **separately for each group**

```
# summarize the grouped data
summarize(
  gapminder,
  mean_lifeExp = mean(lifeExp),
  mean_gdpPercap = mean(gdpPercap),
  .by = continent
)
#> # A tibble: 5 × 3
#>   continent mean_lifeExp mean_gdpPercap
#>   <fct>            <dbl>          <dbl>
#> 1 Asia              60.1          7902.
#> 2 Europe            71.9         14469.
#> 3 Africa            48.9          2194.
#> 4 Americas          64.7          7136.
#> 5 Oceania           74.3         18622.
```

summarize() by group

Combine variables if you want to summarize by more than one group

```
# summarize the grouped data
summarize(
  gapminder,
  mean_lifeExp = mean(lifeExp),
  mean_gdpPercap = mean(gdpPercap),
  .by = c(continent, year)
)
#> # A tibble: 60 × 4
#>    continent  year mean_lifeExp mean_gdpPercap
#>    <fct>     <int>        <dbl>          <dbl>
#>  1 Asia       1952         46.3          5195.
#>  2 Asia       1957         49.3          5788.
#>  3 Asia       1962         51.6          5729.
#>  4 Asia       1967         54.7          5971.
#>  5 Asia       1972         57.3          8187.
#>  6 Asia       1977         59.6          7791.
#>  7 Asia       1982         62.6          7434.
#>  8 Asia       1987         64.9          7608.
#>  9 Asia       1992         66.5          8640.
#> 10 Asia       1997         68.0          9834.
#> # ℹ 50 more rows
```

count()

Counts observations by group

```
# count rows grouped by continent
count(gapminder, continent)
#> # A tibble: 5 × 2
#>   continent     n
#>   <fct>     <int>
#> 1 Africa      624
#> 2 Americas    300
#> 3 Asia        396
#> 4 Europe      360
#> 5 Oceania      24
```

Count by multiple groups:

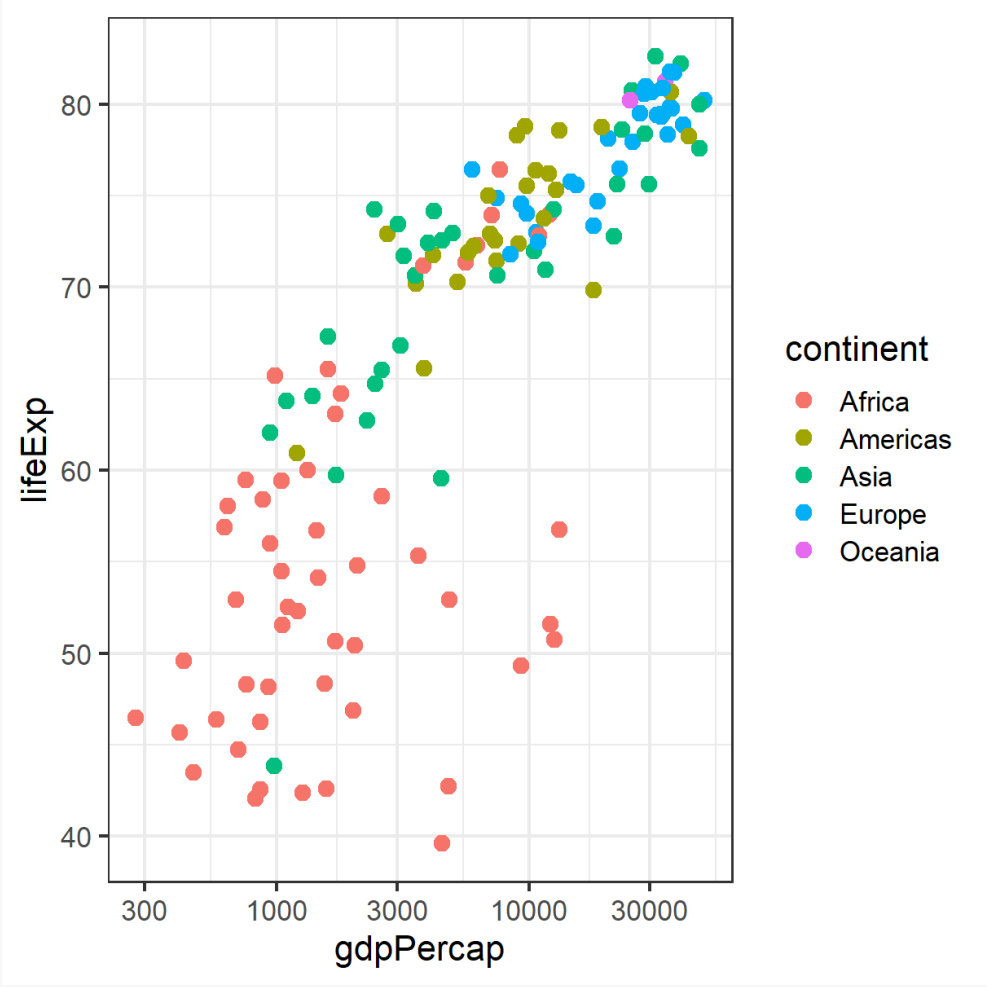
```
# count rows grouped by continent and year
count(gapminder, continent, year)
#> # A tibble: 60 × 3
#>    continent  year     n
#>    <fct>     <int> <int>
#>  1 Africa     1952    52
#>  2 Africa     1957    52
#>  3 Africa     1962    52
#>  4 Africa     1967    52
#>  5 Africa     1972    52
#>  6 Africa     1977    52
#>  7 Africa     1982    52
#>  8 Africa     1987    52
#>  9 Africa     1992    52
```

```
#> 10 Africa     1997    52
#> # ℹ 50 more rows
```

The pipe |> + ggplot

Piping also works well together with `ggplot`

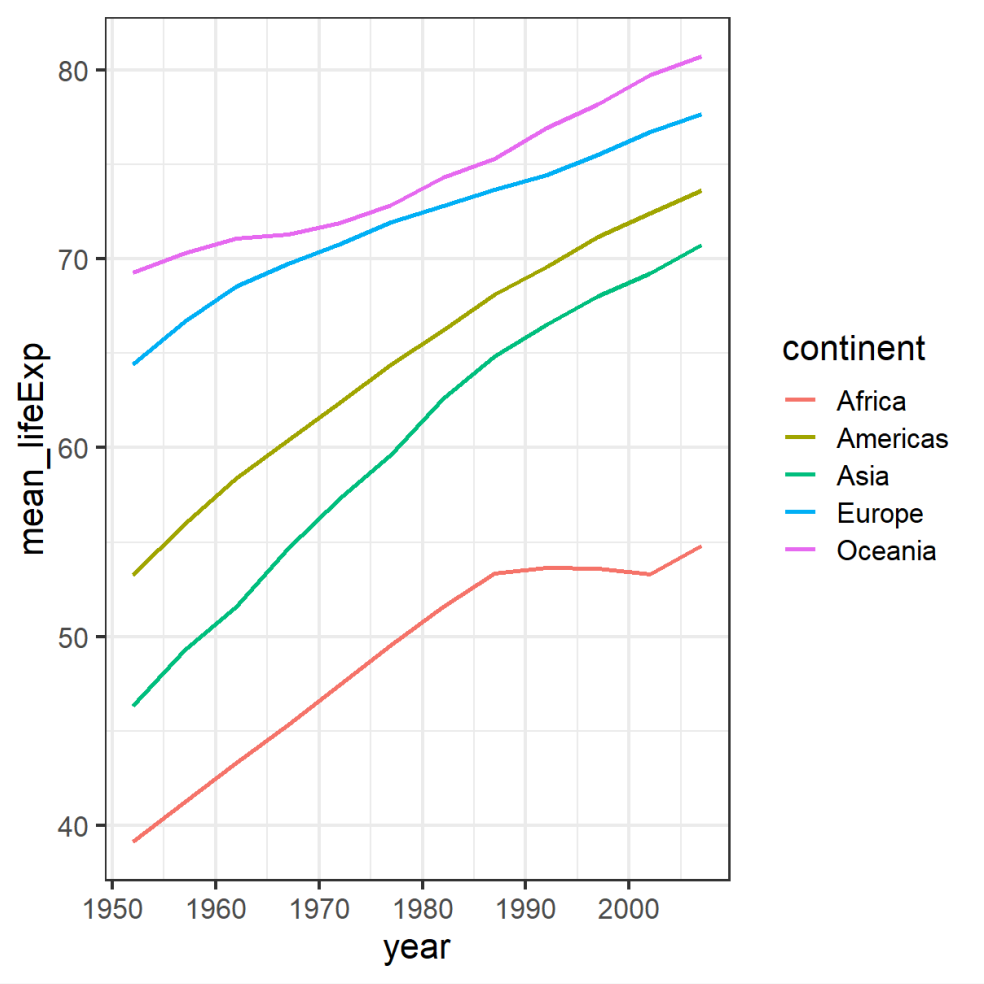
```
gapminder |>
  filter(year == 2007) |>
  ggplot(
    aes(x = gdpPercap, y = lifeExp, color = continent)
  ) +
  geom_point(size = 3) +
  scale_x_log10()
```



The pipe |> + ggplot

You can also combine dplyr operations and ggplot:

```
gapminder |>
  summarize(
    mean_lifeExp = mean(lifeExp),
    .by = c(continent, year)
  ) |>
  ggplot(
    aes(
      x = year,
      y = mean_lifeExp,
      color = continent
    )
  ) +
  geom_line(linewidth = 1)
```



- Use |> between dplyr steps
- Switch to + once you start the ggplot

Now you

Task 2 (30 min)

Summarize data and combine dplyr with ggplot

Find the task description here

Good to know

Good to know

`dplyr` has more useful functions that we didn't cover today:

- **`bind_rows()`**: combine multiple tables with the same columns into one
- **`left_join()`**: combine two tables by a shared column
- **`arrange()`**: sort rows by column values
- **`rename()`**: rename columns

You can find documentation, examples and a cheatsheet for all of these on the dplyr website.

Combine two tibbles by row `bind_rows`

Situation: Two (or more) `tibbles` with the same variables (column names)

```
tbl_a <- gapminder[1:2, ] # first two rows
tbl_b <- gapminder[3:nrow(gapminder), ] # the rest
```

```
tbl_a
```

```
#> # A tibble: 2 × 6
#>   country     continent  year lifeExp     pop gdpPercap
#>   <fct>       <fct>     <int>   <dbl>   <int>     <dbl>
#> 1 Afghanistan Asia       1952    28.8 8425333      779.
#> 2 Afghanistan Asia       1957    30.3 9240934      821.
```

```
tbl_b
```

```
#> # A tibble: 1,702 × 6
#>   country     continent  year lifeExp      pop gdpPercap
#>   <fct>       <fct>     <int>   <dbl>    <int>     <dbl>
#> 1 Afghanistan Asia       1962    32.0 10267083      853.
#> 2 Afghanistan Asia       1967    34.0 11537966      836.
#> # ℹ 1,700 more rows
```

Combine two tibbles by row `bind_rows`

Bind the rows together with `bind_rows()`:

```
bind_rows(tbl_a, tbl_b)
```

```
#> # A tibble: 1,704 × 6
#>   country     continent  year lifeExp     pop gdpPercap
#>   <fct>       <fct>     <int>   <dbl>   <int>     <dbl>
#> 1 Afghanistan Asia       1952    28.8 8425333      779.
#> 2 Afghanistan Asia       1957    30.3 9240934      821.
#> # ℹ 1,702 more rows
```

You can also add an ID-column to indicate which line belonged to which table:

```
bind_rows(a = tbl_a, b = tbl_b, .id = "id")
```

```
#> # A tibble: 1,704 × 7
#>   id    country     continent  year lifeExp      pop gdpPercap
#>   <chr> <fct>       <fct>     <int>   <dbl>    <int>     <dbl>
#> 1 a     Afghanistan Asia       1952    28.8  8425333      779.
#> 2 a     Afghanistan Asia       1957    30.3  9240934      821.
#> 3 b     Afghanistan Asia       1962    32.0 10267083      853.
#> # ℹ 1,701 more rows
```

You can use `bind_rows()` to bind as many tables as you want:

```
bind_rows(a = tbl_a, b = tbl_b, c = tbl_c, ..., .id = "id")
```

Join tibbles with `left_join()`

Situation: Two tables that share some but not all columns.

```
gapminder
```

```
#> # A tibble: 1,704 × 6
#>   country     continent  year lifeExp     pop gdpPercap
#>   <fct>       <fct>     <int>   <dbl>   <int>     <dbl>
#> 1 Afghanistan Asia       1952    28.8 8425333      779.
#> 2 Afghanistan Asia       1957    30.3 9240934      821.
#> # ℹ 1,702 more rows
```

```
# table with more information on the continents
continent_info
#> # A tibble: 5 × 2
#>   continent region
#>   <fct>     <chr>
#> 1 Asia      Global East
#> 2 Europe    Western World
#> 3 Africa    Global South
#> 4 Americas  Western Hemisphere
#> 5 Oceania   Pacific
```

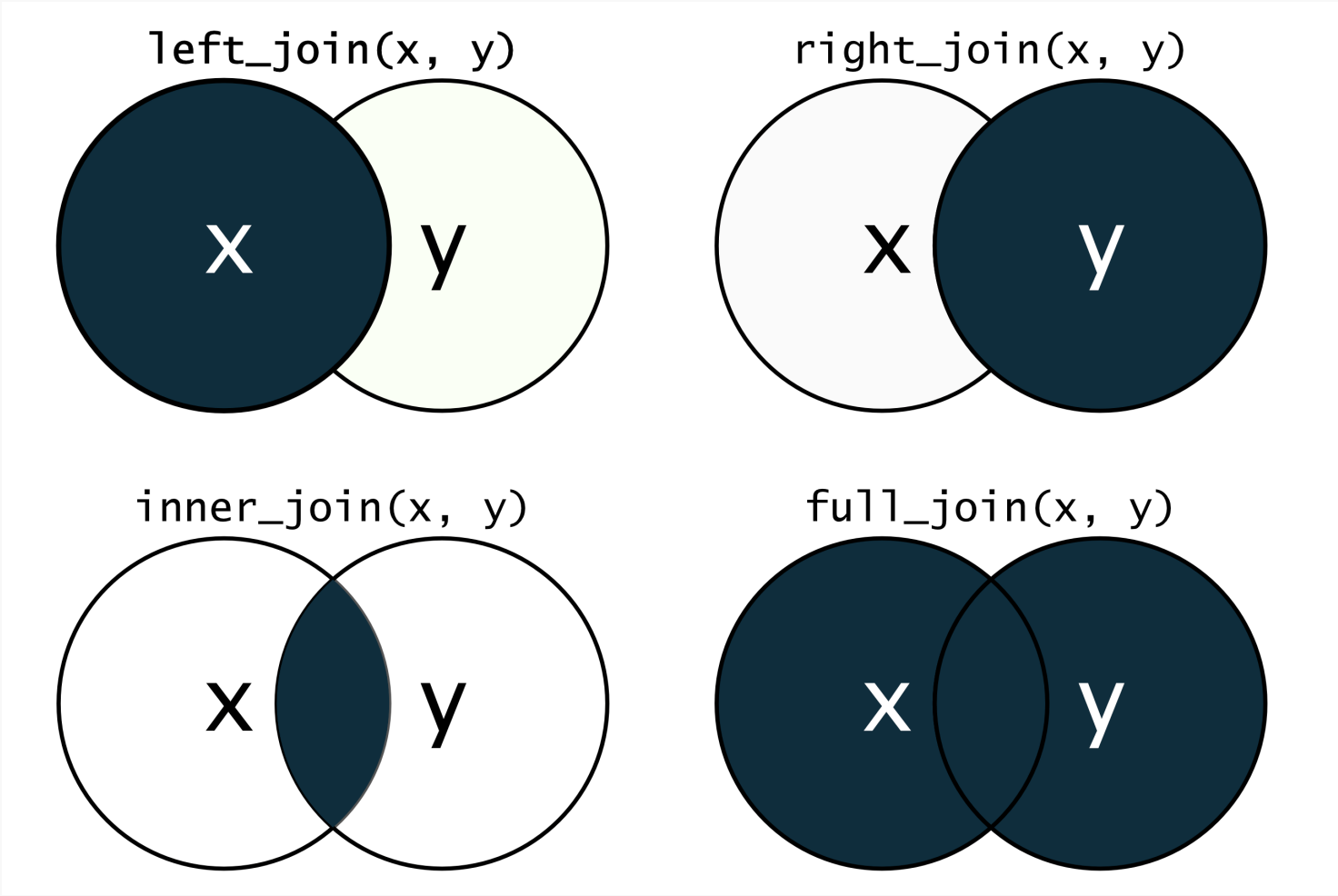
Join tibbles with `left_join()`

Join the two tables by the common column `continent`

```
left_join(gapminder, continent_info, by = "continent")
#> # A tibble: 1,704 × 7
#>    country     continent  year lifeExp      pop gdpPercap region
#>    <fct>       <fct>     <int>   <dbl>    <int>     <dbl> <chr>
#>  1 Afghanistan Asia       1952    28.8  8425333      779. Global East
#>  2 Afghanistan Asia       1957    30.3  9240934      821. Global East
#>  3 Afghanistan Asia       1962    32.0 10267083      853. Global East
#>  4 Afghanistan Asia       1967    34.0 11537966      836. Global East
#>  5 Afghanistan Asia       1972    36.1 13079460      740. Global East
#>  6 Afghanistan Asia       1977    38.4 14880372      786. Global East
#>  7 Afghanistan Asia       1982    39.9 12881816      978. Global East
#>  8 Afghanistan Asia       1987    40.8 13867957      852. Global East
#>  9 Afghanistan Asia       1992    41.7 16317921      649. Global East
#> 10 Afghanistan Asia       1997    41.8 22227415      635. Global East
#> # ℹ 1,694 more rows
```

`left_join()` means that the resulting tibble will contain all rows of `gapminder`, but not necessarily all rows of `continent_info` (in this case it does though).

Different `*_join()` functions



Summary

Data transformation with dplyr

Summary

All `dplyr` functions take a tibble as first argument and return a tibble.

- **`filter()`** / **`filter_out()`**: pick or exclude rows
- **`select()`**: pick columns by name
- **`mutate()`**: add new columns (`case_when()` for conditional values)
- **`drop_na()`**: remove rows with missing values
- **`summarize()`**: collapse rows into summaries (use `.by` for groups)
- **`count()`**: count rows per group
- The pipe **`|>`**: chain operations together, read top to bottom